GUIA PRÁTICO

Guia de uso das skills

Como instalar e usar as 11 skills: o que solicitar, o que cada fluxo entrega e quais decisões continuam nas suas mãos.

A. Preparar o projeto → B. Desenvolver uma funcionalidade → C. Preparar e publicar uma versão

Três momentos

Prepare o projeto com iniciar-projeto e especificar-projeto. Para uma funcionalidade, use implementar-sdd ou as skills individualmente. A publicação é um pedido separado; guardrails e release são opcionais.

Regras do fluxo

Criar ou alterar testes exige pedido ou aceite explícito. Arquivos BDD vazios demonstram a estrutura, mas não ativam o fluxo de cenários. A proteção gerada exige testes existentes e não aceita zero testes coletados como sucesso.

Fonte: README.md, as 11 skills e o modelo AGENTS.md do repositório local. Exemplos de chamados são ilustrativos.

1 • VISÃO GERAL

Como uma skill funciona

Cada skill contém um SKILL.md com nome, descrição e procedimento. Ela consulta o contexto do projeto e aplica as instruções relevantes. Peça pelo nome ou use /nome-da-skill quando o agente aceitar comandos com barra; a seleção automática depende do agente.

AGENTS.md e CLAUDE.md

Instruções de trabalho do agente.

HARNESS.md

Convenção desta coleção para regras específicas do projeto; leia-o conforme a instrução do AGENTS.md.

PRD.md

Produto: problema, escopo, requisitos e regras de negócio.

TDD.md

Technical Design Document: arquitetura, stack, interfaces e configuração. Não significa aqui Test-Driven Development.

DESIGN.md

Estilo visual e componentes, quando há interface.

BDD.md e bdd/

Índice e cenários em Gherkin. Só se considera ativo quando o índice e os cenários estão preenchidos.

BDD é uma prática de colaboração baseada em exemplos de comportamento. Gherkin é uma linguagem para registrá-los; os nomes dos arquivos são convenções desta coleção. IDs dão rastreabilidade sem impor um teste por cenário.

2 • INVENTÁRIO

Preparar e construir

01 iniciar-projeto

Objetivo: Preparar instruções e documentos de referência.

Exemplo: “Use iniciar-projeto na raiz e preserve o que já existe.”

Limite: Não define o produto nem implementa funcionalidades.

02 especificar-projeto

Objetivo: Definir PRD, documento técnico e opções de design e BDD.

Exemplo: “Especifique um sistema de chamados; defina primeiro o problema.”

Limite: Não implementa; não substitui conteúdo existente sem pedido.

03 implementar-sdd

Objetivo: Coordenar especificação, plano e implementação.

Exemplo: “Implemente o encerramento de chamados usando implementar-sdd.”

Limite: Depois oferece testes BDD, revisão e versionamento; são opcionais.

04 especificar-feature

Objetivo: Detalhar uma funcionalidade, regras e critérios.

Exemplo: “Especifique o encerramento de chamados e seus erros.”

Limite: Não implementa; gravação de cenários depende de aprovação.

2 • INVENTÁRIO

Testar, revisar e versionar

05 criar-testes

Objetivo: Gerar testes solicitados para cenários BDD preenchidos e automatizáveis.

Exemplo: “Crie testes para os cenários sem cobertura de bdd/us-02.md.”

Limite: Não altera produção, cenários nem testes existentes; @manual permanece manual.

06 revisar-codigo

Objetivo: Examinar o escopo e sugerir melhorias com impacto concreto.

Exemplo: “Revise o diff e os testes da funcionalidade.”

Limite: Não altera código; avalia SOLID e arquitetura no contexto do projeto.

07 versionar

Objetivo: Agrupar commits, ajustar versão e enviar a branch quando esse fluxo for autorizado.

Exemplo: “Use versionar para organizar, atualizar a versão e enviar a branch.”

Limite: Um pedido apenas de commit não autoriza alterar versão nem fazer push. Não cria tag ou release.

08 gerar-guardrails

Objetivo: Criar ou complementar CI de PR e proteção no GitHub.

Exemplo: “Exija o CI de testes para integrar na principal.”

Limite: Opcional e depende de suíte existente; não faz commit nem push.

2 • INVENTÁRIO

Documentar e publicar

09 atualizar-docs

Objetivo: Sincronizar PRD, TDD e README com o código.

Exemplo: “Atualize a documentação e relate divergências.”

Limite: Preserva edições locais; não altera cenários BDD nem código.

10 publicar-release

Objetivo: Criar e enviar a tag da versão declarada no manifesto.

Exemplo: “Publique a versão e acompanhe o pipeline.”

Limite: Exige árvore limpa; não cria commits nem sobe a versão. Tag enviada não comprova release publicada.

11 gerar-skill

Objetivo: Criar um procedimento abstrato e reutilizável.

Exemplo: “Extraia uma skill de revisão de contratos de API.”

Limite: Não implementa a tarefa descrita; confirma o rascunho antes de gravar.

Escolha rápida: funcionalidade inteira → implementar-sdd
Só requisitos → especificar-feature
Testes ou revisão → criar-testes / revisar-codigo
Commits, versão e envio → versionar

3 • USO EM PROJETO REAL

A. Preparar o projeto

1. Instalar as skills

Na raiz do projeto, execute o comando abaixo. Acrescente -g para instalação global ou -s nome-da-skill para instalar apenas uma. Atualize com npx skills update -y e revise o diff.

```
npx skills add paulocesaaars/skills -a claude-code -a cursor -a github-copilot --copy -y
```

2. Iniciar o projeto

Use /iniciar-projeto. A skill prepara os arquivos de instrução e referência ausentes e verifica as ferramentas. Preserve os arquivos existentes e avalie as opções apresentadas para o modelo de instruções.

3. Especificar o projeto

Use /especificar-projeto. Defina primeiro problema e escopo no PRD; depois stack e arquitetura no documento técnico. A revisão cruzada é opcional; DESIGN.md entra quando houver frontend e os cenários BDD são gerados se você escolher.

Exemplo: “Especifique um sistema para registrar, listar e encerrar chamados; deixe anexos fora da primeira entrega.” Arquivos criados mas vazios não significam que o projeto está especificado.

3 • USO EM PROJETO REAL

B. Desenvolver uma funcionalidade

Especificar → Aprovar o plano → Implementar → Escolher testes, revisão e versionamento

Especificar e implementar

Use /implementar-sdd para coordenar o fluxo ou /especificar-feature para começar manualmente. Informe objetivo, regras, exceções e o que fica fora. Aprove o plano antes da implementação.

Exemplo de comportamento

Para um chamado aberto, quando um atendente autorizado solicita o encerramento, o resultado esperado é que o chamado fique encerrado. Se BDD estiver ativo, a skill pode propor um cenário Gherkin com ID estável; a gravação depende de aprovação.

Testes são uma escolha

O agente só cria ou altera testes quando você pedir ou aceitar a sugestão. No fluxo SDD, a oferta de testes BDD ocorre quando há índice preenchido e cenários em bdd/. Arquivos vazios não ativam essa etapa. Executar testes existentes é uma verificação distinta de criar testes.

Revisão e versionamento

A revisão é opcional e aponta problema, impacto e sugestão. /versionar pode agrupar commits, ajustar versão e enviar a branch; confirme o escopo completo. Um pedido apenas de commit não autoriza automaticamente versão ou push.

3 • USO EM PROJETO REAL

C. Guardrails opcionais

Use /gerar-guardrails se quiser CI e proteção da branch no GitHub. Uma suíte de testes existente é pré-requisito desta skill. Sem testes, ela informa a limitação e não gera a proteção.

Suíte existente → Plano aprovado → CI no PR → Primeira execução verde → Regra de proteção

Validação

O workflow deve executar testes. Nenhum teste coletado deve falhar o job, salvo exceção explícita documentada. A skill verifica comandos e configuração e não faz commit nem push.

Proteção no servidor

A regra depende de permissões e das opções do repositório. Se não houver permissão administrativa, o workflow pode ficar pronto e o pedido para ativação deve ser entregue ao administrador. Não considere a branch protegida antes da confirmação da regra.

Outros hosts e releases

Em outros hosts, a skill fornece um plano em vez de alterar a configuração remota. Se existir workflow de release e você quiser protegê-lo, a skill pode adicionar o portão de testes; não é necessário criar uma release para usar guardrails.

3 • USO EM PROJETO REAL

D. Documentar e publicar

Atualizar a documentação

Use /atualizar-docs para comparar o estado atual com PRD, TDD e README, preservando edições locais. Sem base anterior, a skill faz revisão inicial completa. Divergências pendentes são relatadas e impedem avançar o marcador de revisão.

Preparar a versão

Se escolher o fluxo completo, /versionar organiza commits, avalia a versão e envia a branch. Uma solicitação de apenas commit não inclui automaticamente subir versão ou enviar. A versão no manifesto é usada pela skill de publicação.

Publicar a release

Use /publicar-release com árvore limpa e versão válida no manifesto. A skill confere as condições, apresenta o plano e cria e envia a tag. Para a primeira release, considera o histórico alcançável mesmo sem tag anterior.

Distinguir os resultados

Tag é uma referência Git. Release é uma versão publicada. Deploy é a implantação em um ambiente. A tag pode iniciar um pipeline, mas sua execução não prova que a publicação ou o deploy terminou. Confira o estado e os artefatos antes de declarar sucesso.

SemVer trata compatibilidade da API pública; tipos de Conventional Commits são indícios. Considere versões 0.y.z e pré-lançamentos conforme a política do projeto.